

05. 開発・運用ガイド

Version 1.0 (FIX) / 2026-08-17

本書は、開発着手から実験導入までに必要な環境構築・規約・デプロイ・店舗運用の手順をまとめたものである。

1. 開発環境

項目	バージョン / ツール	備考
Node.js	20.x LTS	フロントエンド開発用
Python	3.12	バックエンド開発用
パッケージ管理	npm / pip (venv)	追加ツールは導入しない(構成を単純に保つため)
エディタ	任意	Prettier / Ruff のフォーマッタ連携を推奨
DB	Supabase (クラウド)	ローカルDBは立てず、開発用プロジェクトを1つ作成して共用する

ローカルDBを立てない理由
実験導入・個人開発の規模では、ローカルにDockerでPostgreSQLを立てる手間とトラブルの方がコストが高い。Supabaseの無料プロジェクトを「開発用」「本番用」の2つ作成し、環境変数で切り替える方式とする。

2. リポジトリ構成

```
hihumi_manage/
├── docs/           設計ドキュメント一式(本書を含む)
├── db/
│   ├── schema.sql  DDL([02]5章と同内容)
│   └── seed.sql     初期データ([02]6章と同内容)
├── frontend/       React + Vite ([03]8章参照)
├── backend/        FastAPI
│   ├── app/
│   │   ├── main.py  エントリポイント・ルータ登録
│   │   ├── config.py  環境変数の読み込み
│   │   ├── db.py     DBセッション管理
│   │   ├── models.py  SQLAlchemyモデル
│   │   ├── schemas.py Pydanticスキーマ
│   │   ├── deps.py    認証(Bearer/Session/Cron)の共通依存
│   │   ├── errors.py  共通エラーハンドラ
│   │   └── routers/
│   │       ├── customer.py  顧客向けAPI
│   │       ├── staff.py     スタッフ向けAPI
│   │       └── jobs.py       バッチAPI
│   ├── tests/
│   ├── requirements.txt
│   ├── .env
│   └── README.md
```

3. 環境変数一覧

3.1 Backend (Render)

変数名	用途	例 / 備考
DATABASE_URL	SupabaseのPostgreSQL接続文字列	postgresql://postgres:***@db.***.supabase.co:5432/postgres
SUPABASE_SERVICE_ROLE_KEY	Service Role Key(RLSをbypassする鍵)	外部に絶対に漏らさない
JWT_SECRET	スタッフJWTの署名鍵	十分な長さのランダム文字列
JWT_EXPIRE_HOURS	JWTの有効期限(時間)	8
CRON_SECRET	バッチAPIの共有シークレット	ランダム文字列
ALLOWED_ORIGINS	CORS許可オリジン	https://<本番FEドメイン>

3.2 Frontend (Vercel)

変数名	用途	例 / 備考
VITE_API_BASE_URL	Backend APIのベースURL	https:///api/v1

鍵の取り扱い

SUPABASE_SERVICE_ROLE_KEY はRLSを無効化できる最強の鍵であり、これが漏れると全データが読み書き可能になる。Backendの環境変数にのみ設定し、フロントエンドのコード・リポジトリ・ドキュメントには絶対に記載しない。

.env ファイルは必ず .gitignore に追加し、コミットしないこと。

4. セットアップ手順

#	手順	内容
1	Supabaseプロジェクト作成	開発用プロジェクトを作成し、DATABASE_URL と Service Role Key を控える。
2	スキーマ構築	SQL Editorで db/schema.sql を実行する。
3	初期データ投入	db/seed.sql を実行する(卓7件・商品サンプル)。
4	スタッフ登録	5章の手順でパスワードハッシュを生成し、staffテーブルへINSERTする。
5	Backend起動	backend/ で venv作成 → pip install -r requirements.txt → .env作成 → uvicorn app.main:app --reload
6	Frontend起動	frontend/ で npm install → .env.local作成 → npm run dev
7	疎通確認	http://localhost:8000/docs でAPI一覧、http://localhost:5173 で画面を確認する。

5. スタッフパスワードの登録

パスワードは平文で保存しない。以下でbcryptハッシュを生成し、その値をstaffテーブルに投入する。

```
# backend/ の venv 内で実行
python -c "import bcrypt; print(bcrypt.hashpw(b'実際のパスワード', bcrypt.gensalt()).decode())"

# 出力された $2b$12$... を seed.sql の password_hash に設定してINSERT
```

実験導入時はスタッフ全員で1アカウントを共用する運用でも差し支えない(V1では権限差がないため)。将来的に個人別の操作履歴が必要になった場合は、staffレコードを人数分作成する。

6. コーディング規約(最小限)

規約は「守らないと不具合や手戻りに直結するもの」に絞る。書式の統一はフォーマッタに任せ、人手でのレビュー対象としない。

分類	規約	理由
共通	書式はフォーマッタ(Prettier / Ruff)に一任し、手動整形しない	レビューを設計・ロジックに集中させるため
共通	環境依存の値(URL・鍵・卓座標)をコードに直書きしない	環境切替時の事故と、DBとの二重管理を防ぐ
FE	税込計算は shared/price.ts の関数のみを使う	計算式の散在による表示金額の不一致を防ぐ
FE	dangerouslySetInnerHTML を使用しない	sessionStorage上のsession_tokenをXSSから守るため
FE	サーバー状態はSWRのキャッシュを唯一の情報源とし、Zustandに複製しない	状態の二重管理による不整合を防ぐ
BE	認証・セッション検証は必ずDependsで共通化する	エンドポイント単位の実装漏れが重大な脆弱性に直結するため
BE	複数レコードの更新は必ずトランザクション内で行う	途中失敗時の中途半端なデータを防ぐ
BE	ステータス遷移の妥当性は必ずサーバー側で検証する	クライアントの表示制御だけでは不正な遷移を防げないため
DB	スキーマ変更は db/schema.sql に反映し、変更内容を[02]にも記載する	ドキュメントと実装の乖離を防ぐ

7. デプロイ手順

対象	手順
Database (Supabase)	本番用プロジェクトを作成し、schema.sql / seed.sql を実行する。開発用とは別プロジェクトとし、接続文字列を混同しないよう管理する。
Backend (Render)	GitHubリポジトリを連携し、Root Directory を backend に設定。Build: pip install -r requirements.txt / Start: uvicorn app.main:app --host 0.0.0.0 --port \$PORT。3章の環境変数を設定する。
Frontend (Vercel)	Root Directory を frontend に設定。Framework Preset は Vite。VITE_API_BASE_URL を設定する。
Cron (Vercel)	vercel.json に日次スケジュールを定義し、POST /api/v1/jobs/daily_clear を X-Cron-Secret 付きで呼び出す。

デプロイ後に必ず確認すること

Backendの ALLOWED_ORIGINS

に本番フロントエンドのドメインが設定されているか(未設定だとCORSで全滅する)。

Supabaseのテーブル一覧で、全テーブルのRLSが有効(Enabled)になっているか。

ブラウザの開発者ツールで、フロントエンドからSupabaseへの直接通信が発生していないか。

8. QRコードの作成・運用

#	手順
1	卓ごとのURLを用意する。例: https://<本番ドメイン>/?table=1 (T1) ~ ?table=7 (C3)。table の値は tables.id であり、[02]6.1節の一覧を参照する。
2	各URLのQRコードを生成する(任意のQRコード生成ツールで可)。
3	卓のラベル(T1 / C1 等)を併記して印刷し、ラミネート加工のうえ各卓に設置する。
4	設置後、実際にスマートフォンで読み取り、正しい卓のメニューが開くことを7卓すべてで確認する。

QRコードは貼り替え不要

QRコードに埋め込むのは卓のIDのみであり、session_tokenは含まれない。そのため開栓のたびにQRコードを作り直す必要はなく、一度設置すれば貼り替え不要である。セッションの正当性は、開栓時に再発行されるsession_tokenによってサーバー側で検証される。

9. 日次運用チェックリスト

タイミング	確認事項
開店前	スタッフ端末でダッシュボードにログインできること。全卓が「空席」であること。
開店前	当日提供できない商品を品切れ設定にしておく(STF-004)。
営業中	客からの呼出(赤ドット)に対応漏れがないか、定期的にダッシュボードを確認する。
閉店時	全卓を「会計クリア」して空席に戻す。
翌営業日	クリア漏れがあった場合、日次バッチで自動クリアされている。必要に応じて前日の伝票内容を確認する。

10. トラブル時の対処

症状	想定原因と対処
客の画面に「ご利用いただけません」と出る	卓が未開栓。ダッシュボードから該当卓を開栓する。
客の画面が「QRコードを再読み込みしてください」となる	会計クリア済み、または別端末で開栓し直された。再度QRコードを読み取ってもらう。
開栓しようすると「既に関栓済み」と出る	他のスタッフが先に開栓している。画面を再読み込みして確認する。
注文がダッシュボードに出てこない	最大4秒の反映遅延がある。それ以上続く場合はネットワークとBackendの稼働を確認する。
スタッフ画面がログイン画面に戻る	JWTの有効期限(8時間)切れ。再ログインする。
最初のアクセスだけ極端に遅い	Renderのコールドスタート。仕様上許容範囲([00]NFR-05)。

11. 変更履歴

版	主な変更内容
V1.0 (FIX)	新規作成。開発環境・リポジトリ構成・環境変数・セットアップ手順・コーディング規約・デプロイ・QR運用・日次チェックリスト・トラブル対処を定義。